



DEPLOYING

Two halves, two very different places.

	ETSCH	GLITZY STUDIO
Where	Cloudflare Workers (static assets)	A host you control
Public	Configured, not published	No, and deliberately so
Needs	wrangler	Python 3.11+, NumPy, Pillow, ffglitch

Nothing is published to Cloudflare yet. `wrangler.toml` is complete and `wrangler deploy --dry-run` passes; the deploy has simply not been run.

Worth stating plainly, because "deploy the app" and "expose my infrastructure" are easy to conflate and only one of them is happening here: **publishing Etsch exposes no server**. It is static assets with no Worker code, and it makes no backend calls – the only `fetch()` in the whole bundle is a relative, same-origin `/handoff/<id>`. That route exists on the internal deployment and does not exist on Cloudflare, where it 404s and the app reports a miss and carries on. There is no tunnel, no port-forward, and no reference to any private host in `etsch/public` – no LAN address, no internal host-name, no port. Grep it before you take that on trust:

SH

```
| grep -rnE '10\.0\.\|\.hq\.|localhost|8090' etsch/public/ # expect no matches
```

SEC.01

ETSCH → CLOUDFLARE

`wrangler.toml` declares a Worker named `glitzy` with an `assets` directory and **no `main`**. That combination means Cloudflare serves `etchs/public` directly: no script runs on a request, so there is nothing billed per request and no runtime to keep patched.

SH

```
| npm ci
| npx wrangler deploy --dry-run # validates the config, no credentials needed
| npx wrangler deploy
```

CREDENTIALS

Either works:

- ▶ `npx wrangler login` — OAuth in a browser, nothing long-lived stored.
- ▶ `CLOUDFLARE_API_TOKEN` — for a headless box. The token needs **Account → Workers Scripts → Edit**, **Account → Account Settings → Read** (wrangler discovers the account id with it), and **Zone → DNS → Edit** on the zone, so it can write the custom domain's record.

A token scoped only to `Zone → DNS → Edit` — the kind an ACME client uses for DNS-01 — **is not enough and fails in a confusing way**: it authenticates fine, `user/tokens/verify` says `active`, and then `accounts` comes back as an empty list rather than an error, so the failure surfaces as "could not determine account" rather than "insufficient permissions".

THE CUSTOM DOMAIN

`[[routes]] custom_domain = true` in `wrangler.toml` attaches `glitzy.ripostelabs.xyz` on deploy, and Cloudflare writes the DNS record itself – the zone is on the same account. Keeping the hostname in the config rather than in the dashboard means a fresh account can be rebuilt from this file alone.

The record it writes is a proxied AAAA to `100::`, which is the documented placeholder for a Workers custom domain. It looks wrong in the DNS list and is not.

CHECKING IT

SH

```
curl -sI https://glitzy.ripostelabs.xyz/ | head -3
curl -s https://glitzy.ripostelabs.xyz/ | grep -o '<title>[^<]*'
```

A 200 with the right title is the whole check. There is no server to be unhealthy.

SEC.02

RUNNING THE STUDIO SOMEWHERE ELSE

`studio/deploy/provision.sh <user@host>` provisions the studio onto a fresh Ubuntu 22.04+ / Debian 13 host, **x86-64 or arm64**, sizing the unit from that host's own RAM and disk. It binds to `127.0.0.1` unless told otherwise, because the engine has no authentication of its own.

Requirements, and the remote-access options including the Tailscale route that needs no new machine at all, are in `studio/deploy/REMOTE-ACCESS.md` RL-360-REMOTE-ACCESS. The short version: **2 GB RAM is the floor** – one render peaks near 900 MB, and below a ~1.4 GB ceiling the cgroup kills the engine, which looks like a 502.

SEC.03

THE STUDIO → A HOST YOU CONTROL

The studio is **not** on the public internet and should not be. It accepts file uploads, spawns `ffgac/ffedit` subprocesses on them, and holds a cache keyed on user input. It belongs behind an authenticating reverse proxy on a network you trust. `studio/deploy/` holds the site file for that proxy.

SH

```
studio/deploy.sh --restart
```

That copies the tree to the engine host and restarts `glitzyd.service`. See `studio/README.md` `RL-360-STUDIO` for what the engine needs installed.

MEMORY IS THE OPERATIONAL CONSTRAINT

The unit sets `MemoryMax`. This is load-bearing: a single numpy operation on a large clip can allocate more than a gigabyte in about two seconds, and the costliest operation measures **about 104 bytes per pixel** – not the 4 bytes an RGBA frame suggests, and not the 75 B/px this document claimed until 2026-08-15. The binding op is `colour.hsv`, not the warp family; the warps are cheaper per pixel but carry a separate ~410 MB cost per **frame** at the 1600×1600 maximum. Both models, and the measurements behind them, are written out in `studio/glitzyd/clip.py`.

Raising the container's memory does **not** raise this ceiling. `MemoryMax` on the unit is a second, lower ceiling, and it is the one that kills renders; the two have to be raised together, along with `GLITZY_MAX_PIXELS`.

When `MemoryMax` is hit the whole cgroup is **SIGKILLED**, so the engine dies rather than returning an error, and a reverse proxy in front of it reports a **502**. A 502 from the studio means the engine was killed, not that it hung. `journalctl -u glitzyd` shows the `oom-kill` line and the peak.

Because **SIGKILL** is not catchable, anything that would exceed the cap has to be refused **before** it allocates. That is what `clip.check_budget()` and `MAX_PIXELS` are for; they are not decoration, and raising them without measuring bytes-per-pixel per operation first will just move the crash.

RENAMING AN EXISTING DEPLOYMENT

If you are moving a deployment that predates the Glitchsheet → Glitzy rename, **move the data directory, do not just point at a new one:**

SH

```
systemctl stop glitchd2
mv /var/lib/glitchsheet2 /var/lib/glitzy
mv /opt/glitchsheet2      /opt/glitzy
```

The store is content-addressed and its export links are meant to be permanent – that is the whole reason it keys files on `sha1(bytes)` and evicts nothing. Starting the renamed service against an empty `/var/lib/glitzy` does not error; it comes up clean and every previously-shared link 404s with “that export has expired”. The service name changes from `glitchd2` to `glitzyd`; the **v1** engine’s unit is still `glitchd` and should be left alone, since it is the rollback.

Check the move worked by the cache, not by the service starting:

SH

```
curl -s localhost:8090/api/health # cache.entries must be what it was, not 0
```

KEEPING THE OLD HOSTNAMES ANSWERING

`studio/deploy/160-glitzy.caddy` and `165-etsch.caddy` each match **two** hostnames – the new name and the pre-rename one – so nothing anybody has bookmarked breaks.

They are aliases, not redirects, and that is deliberate: the studio sends the browser to Etsch with the sheet id in a **URL fragment** (`#handoff=<id>`), and fragments are never sent to the server. A 301 from the old name to the new one would arrive with no fragment to put back, so the handoff would land on an empty sheet with no error anywhere.

Both names must also be listed in Authelia’s access-control rule. Authelia matches the **first** rule whose domain matches; an alias that is not listed does not inherit the rule, it falls through to the default policy.

SOURCE DEPLOY.tex SHA 866b31983b6c BUILT 2026-09-07